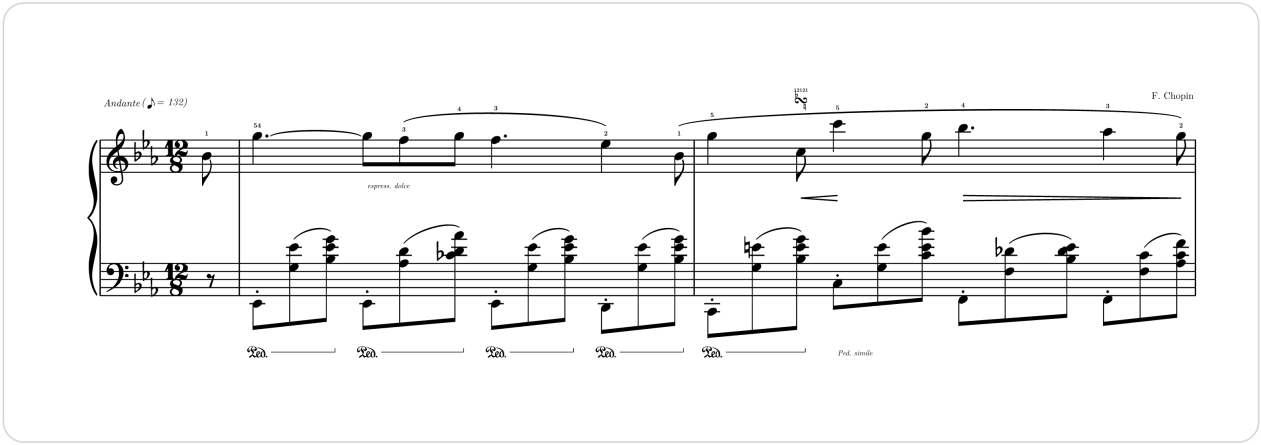
typed-scores

Western music notation, natively in Typst

User Guide



Version 0.6.0

Contents

1 Introduction 2
2 Getting started 3
2.1 Installation and import 3
2.2 Quick measure with `bar()` 3
2.3 Light and dark notation 3
3 Scores and systems 5
3.1 One staff 5
3.2 Multiple staves 5
3.3 Independent voices 6
3.4 Bar metadata 8
3.5 Percussion staves 11
3.6 Harmony symbols 12
3.7 Figured bass 13
3.8 Lyrics 14
3.9 Barlines, navigation, and labeled endings 15
3.10 Score arguments 18
3.11 Indentation and justification 19
3.12 Transposition and parts 20
4 Event language 23
4.1 Duration inheritance 23
4.2 Pitch spelling and relative octaves 23
4.3 Relative chords 24
4.4 Beam boundaries 24
4.5 Tuplets 24
4.6 Grace notes 25
4.7 Cue-sized notes 26
4.8 Tremolos and arpeggios 27
4.9 Expressive annotations 28
4.10 Ottava brackets 29
4.11 Cross-staff notation 30
5 Guitar notation 35
5.1 Guitar clefs 35
5.2 Tablature staves 35
5.3 Chord diagrams 38
6 Importing MusicXML and ABC 41
7 Validation and layout 43
7.1 Worked example: Beethoven 43
7.2 Worked example: Chopin 44
8 Release showcase 46
9 Limitations 48
9.1 Current scope 48
9.2 Grammar at a glance 48
9.3 License and credits 49
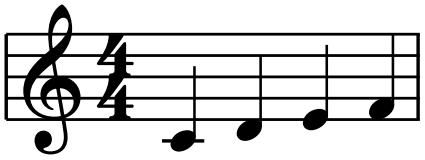
1 Introduction

`typed-scores` turns compact musical text into engraved western notation inside Typst. A Rust/WASM plugin parses pitches, durations, exact onsets, and beam groups; Typst and CeTZ lay out the score with bundled Bravura glyphs.

The package is designed for lecture notes, theory worksheets, analytical examples, short compositions, and reusable excerpts whose musical content should remain readable and versionable as text.

> **Note.** The public API is intentionally small: `score` for complete scores, `bar` for a quick one-staff measure, and `chord-diagram` for a standalone fretboard diagram, with the `guitar-chords` shape library. `part` extracts a staff; `import-score` and `read-score` import existing music.

```typst
#score(
  time: "4/4",
  bars: ((notes: "c4:q d e f"),),
)
```



2 Getting started

2.1 Installation and import

Import the package from the Typst preview namespace:
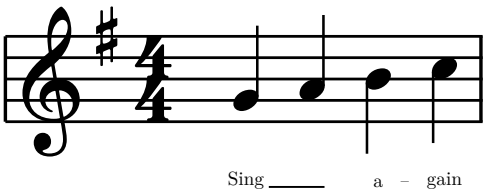
```typst
#import "@preview/typed-scores:0.6.0": *
```

The examples in this guide use lowercase pitch letters for fast entry. Uppercase remains accepted with identical pitch and octave semantics.

2.2 Quick measure with `bar()`

`bar` is a convenience wrapper over the same score pipeline used by `score`. It accepts a note string, optional lyrics, and its clef, key, and optional meter.

```typst
#bar(
  "g4:q a b c5",
  lyrics: "Sing __ a -- gain",
  clef: "treble",
  key: "G",
  time: "4/4",
)
```



With `time` omitted, the bar remains useful for a short fragment and does not perform full-measure duration validation.
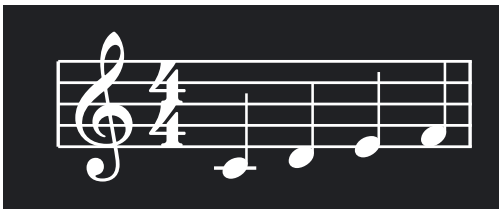
Argument	Type	Default	Description
`notes`	`str`	required	One event string.
`lyrics`	`str | array | none`	`none`	One lyric verse or an array of verses.
`clef`	`str`	`"treble"`	Treble, bass, alto, tenor, or percussion clef.
`key`	`str`	`"C"`	Major or minor key signature.
`time`	`str | none`	`none`	Meter and expected duration, such as `"4/4"` .
`theme`	`str`	`"auto"`	Notation ink: `light` uses black, `dark` uses white, and `auto` follows the surrounding solid text fill.
`lyric-size`	`number`	`0.9`	Lyric text size in staff spaces.
`lyric-font`	`str | none`	`none`	Optional font family for lyrics.
`lyric-gap`	`number`	`0.8`	Clearance above the first verse.
`verse-gap`	`number`	`1.45`	Baseline distance between verses; must be at least `lyric-size`.

2.3 Light and dark notation

Use `theme: "dark"` when placing a score on a dark surface. It recolors staff geometry, text, and every bundled Bravura glyph together. The default `theme: "auto"` follows the surrounding Typst text fill; it cannot inspect CSS changes applied by an HTML host after Typst has exported the SVG.

```typst
#block(fill: rgb("#202124"), inset: 10pt)[
  #bar(
    "c4:q d e f",
    time: "4/4",
    theme: "dark",
  )
]
```

Typst
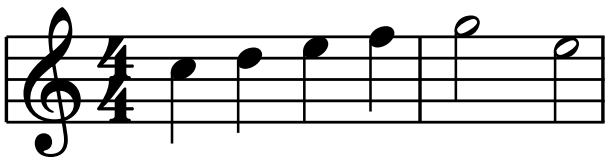


3 Scores and systems

Every score is an ordered `bars:` list. Each bar is a dictionary: it contains staff event strings and optional metadata that belongs to that musical bar. Never place structural bar delimiters inside an event string.

3.1 One staff

For one staff, omit `staves:` and use the reserved `notes:` field in every bar. There is no artificial staff name to repeat. A string is one rhythmic voice; an array of two to four strings creates independent simultaneous voices on the same staff.

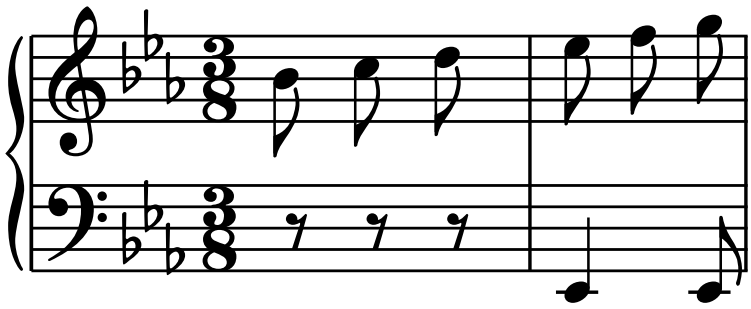
```typst
#score(
  clef: "treble",
  time: "4/4",
  bars: (
    (notes: "c5:q d5:q e5:q f5:q"),
    (notes: "g5:h e5:h"),
  ),
)
```



3.2 Multiple staves

For two or more staves, declare a map of permanent staff IDs and clefs. Every bar must then contain exactly one string or voice-string array for each declared ID.

```typst
#score(
  staves: (
    upper: (clef: "treble"),
    lower: (clef: "bass"),
  ),
  key: "Eb",
  time: "3/8",
  bars: (
    (
      upper: "bb4:e c5:e d5:e",
      lower: "r:e r:e r:e",
    ),
    (
      upper: "eb5:e f5:e g5:e",
      lower: "eb2:q eb2:e",
    ),
  ),
  beams: true,
)
```

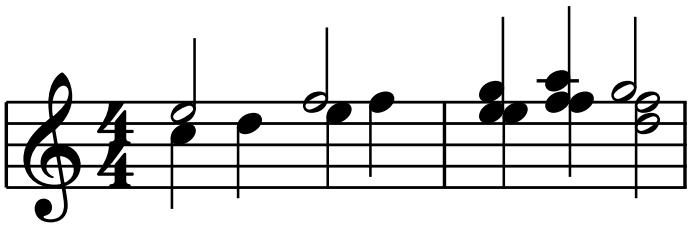


Staff IDs are Typst dictionary keys, so keep them short and machine-oriented. They cannot reuse reserved bar fields such as `notes`, `lyrics`, `key`, `time`, `partial`, `tempo`, `harmony`, or `figures`.

3.3 Independent voices

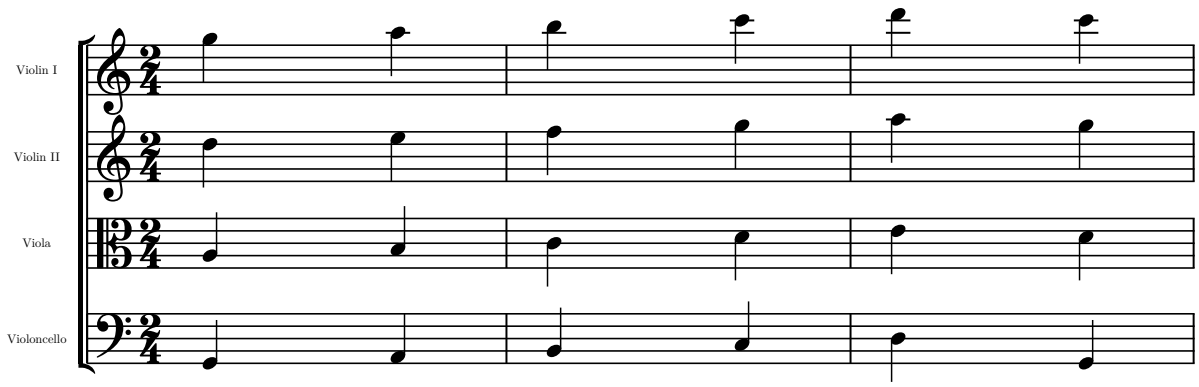
Use an array when notes on one staff have independent rhythms. The voice count must stay constant for that staff throughout the score. Following LilyPond's standard polyphonic convention, the first voice uses upward stems, the second downward stems, and later voices alternate. Rests move into separate vertical lanes. Coincident identical noteheads merge; unisons and seconds that cannot merge are shifted horizontally while retaining their shared onset.

```typst
#score(
  time: "4/4",
  bars: (
    (notes: (
      "e5:h f5:h",
      "c5:q d5:q e5:q f5:q",
    )),
    (notes: (
      "(e5 g5):q (f5 a5):q g5:h",
      "e5:q f5:q (d5 f5):h",
    )),
  ),
)
```



Each staff may also define `label` for its first-system name and `short-label` for the optional name on later systems. This follows LilyPond's `instrumentName` / `shortInstrumentName` behavior: names live in a shared column to the left of the group, center horizontally within that column, and center vertically on their individual staff. The compact example below shows the full first-system names; wrapped systems substitute the short names.

```typst
#score(
  staves: (
    violin-one: (clef: "treble", label: "Violin I", short-label: "Vln. I"),
    violin-two: (clef: "treble", label: "Violin II", short-label: "Vln. II"),
    viola: (clef: "alto", label: "Viola", short-label: "Vla."),
    cello: (clef: "bass", label: "Violoncello", short-label: "Vc."),
  ),
  group: "bracket",
  time: "2/4",
  scale: 0.58,
  ragged-right: false,
  bars: (
    (violin-one: "g5:q a5:q", violin-two: "d5:q e5:q", viola: "a3:q b3:q", cello: "g2:q a2:q"),
    (violin-one: "b5:q c6:q", violin-two: "f5:q g5:q", viola: "c4:q d4:q", cello: "b2:q c3:q"),
    (violin-one: "d6:q c6:q", violin-two: "a5:q g5:q", viola: "e4:q d4:q", cello: "d3:q g2:q"),
  ),
)
```



The `group` argument controls the symbol at the left of a multi-staff system: `"brace"`, `"bracket"`, `"line"`, or `"none"`. Its default, `auto`, uses no symbol for one staff, a brace for two staves, and a bracket for larger ensembles. Brace groups connect barlines across their staves; the other styles keep each staff's barline separate.

3.4 Bar metadata

`clef`, `key`, `time`, and `tempo` may be set on any bar and persist until changed. `partial` changes duration validation only: it does not change the displayed meter. When a key change removes old alterations, cancellation naturals are engraved before the replacement key signature. This replaces separate section or parallel-array forms.
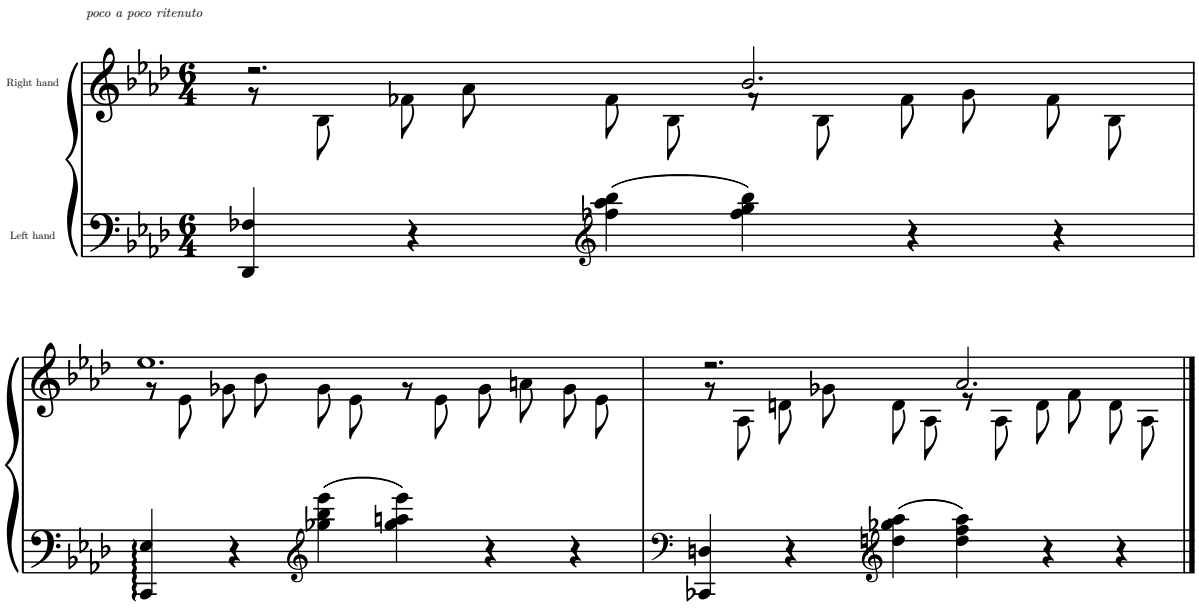
For a single staff, a clef change is `clef: "bass"`. For multiple staves, use a dictionary containing only the staves that change, such as `clef: (left: "bass")`. As in LilyPond, a mid-system change is reduced in size; the active clef is printed full-size when a new system begins.

To change clef within a bar, write `clef treble` in that staff's event string. It takes effect before the next event and remains active in later bars until a bar-level or inline change replaces it. Inline changes need one voice on the staff, must be written while the voice is on its home staff, and must be followed by a home-staff event. Place grace groups before the inline clef command. Use bar metadata for changes at the start of a bar and for changes to or from tablature or percussion.
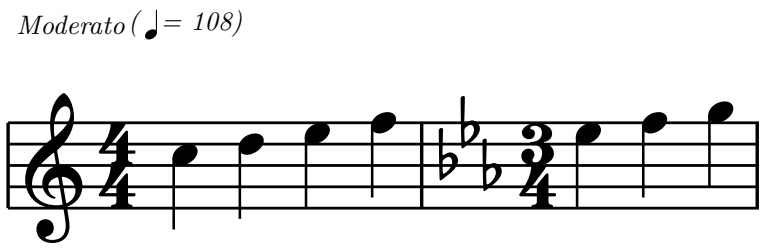
This excerpt is measures 73–75 of Liszt's **Liebestraum No. 3**, in 6/4. The left-hand staff returns to bass clef at each barline, then switches to treble for the two chords. The public-domain MusicXML transcription is available in the MuseTrainer score library.

Typst

```
#score(
  staves: (upper: (clef: "treble", label: "Right hand"), lower: (clef: "bass", label: "Left hand")),
  key: "Ab", time: "6/4", scale: 0.5,
  bars: (
    (
      tempo: "poco a poco ritenuto",
      upper: ("r:h. bb4:h.", "r:e bb3:e fb4:e ab4:e fb4:e bb3:e r:e bb3:e fb4:e g4:e fb4:e bb3:e"),
      lower: "(db2 fb3):q r:q clef treble (fb5 ab5 bb5):q[s1(] (fb5 g5 bb5):q[s1)] r:q r:q",
    ),
    (
      clef: (lower: "bass"),
      upper: ("eb5:w.", "r:e eb4:e gb4:e bb4:e gb4:e eb4:e r:e eb4:e gb4:e a4:e gb4:e eb4:e"),
      lower: "(c2 eb3):q[arpeggio] r:q clef treble (gb5 bb5 eb6):q[s1(] (gb5 a5 eb6):q[s1)] r:q r:q",
    ),
    (
      clef: (lower: "bass"),
      upper: ("r:h. ab4:h.", "r:e ab3:e d4:e gb4:e d4:e ab3:e r:e ab3:e d4:e f4:e d4:e ab3:e"),
      lower: "(cb2 d3):q r:q clef treble (d5 gb5 ab5):q[s1(] (d5 f5 ab5):q[s1)] r:q r:q",
      barline: (right: "final"),
    ),
  ),
)
```



```typst
#score(
  clef: "treble",
  key: "C",
  time: "4/4",
  bars: (
    (
      notes: "c5:q d5:q e5:q f5:q",
      tempo: (text: [Moderato], beat: "quarter", bpm: 108),
    ),
    (
      key: "Eb",
      time: "3/4",
      notes: "eb5:q f5:q g5:q",
    ),
  ),
)
```



For a metronome mark, use a tempo dictionary rather than inserting a Unicode note character. `beat` accepts `whole`, `half`, `quarter`, `eighth`, `sixteenth`, or `thirty-second`, and the matching note glyph is drawn automatically. `text` is optional, so `(beat: "eighth", bpm: 132)` produces just the note-and-number mark.

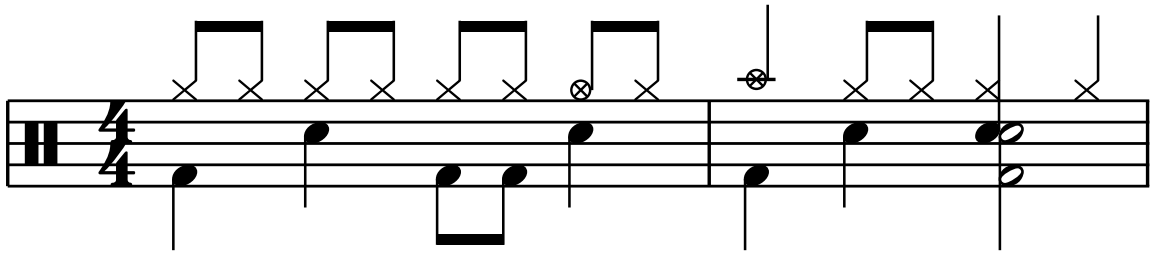
3.5 Percussion staves

`clef: "percussion"` draws the neutral percussion clef. Its lines and spaces read as a treble staff's do, so each instrument is written at the pitch of its line or space. Drum legends differ between publishers; in the common Weinberg layout the bass drum is F4, the snare C5, the hi-hat G5, the ride F5, and the crash A5, while LilyPond's default drum style puts the hi-hat on E5 and the crash on G5. The notes are unpitched, so they take no accidentals, and the staff carries no key signature even in a score whose other staves do; its time signatures stay aligned with theirs. Percussion staves accept everything else a staff does, including independent voices, beams, articulations, and clef changes.

The notehead shape tells the player what kind of instrument is struck: round heads for drums, X heads for cymbals and hi-hat, and a circled X for a crash or an open hi-hat. Write `[x]` or `[circle-x]` after a note, or after a chord to change every head. Inside a chord, the same block after one pitch changes only that head, so a hi-hat struck with the snare is `(g5[x] c5):q` . A shape written on a note belongs to that event only.

A drum part repeats the same instruments throughout, so a staff can carry a drum map instead: `heads` pairs written pitches with `"x"` or `"circle-x"` , and every note drawn on that staff at a listed pitch takes that head. Give it to a declared staff beside its clef, or to the score itself for the single-staff form, as with `clef` . A shape written on the note still wins, and `[normal]` restores a round head at a mapped pitch.

```typst
#score(
  clef: "percussion",
  heads: (g5: "x", a5: "circle-x"),
  time: "4/4",
  beams: true,
  bars: (
    (notes: (
      "g5:e g g g g g g[circle-x] g",
      "f4:q c5 f4:e f c5:q",
    )),
    (notes: (
      "a5:q g5:e g (g5 c5):q g5",
      "f4:q c5 (f4 c5):h",
    )),
  ),
)
```

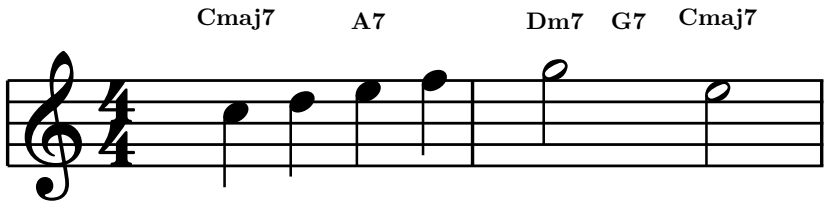


X heads work on any staff, for example for spoken notes. Stems attach where each Bravura glyph expects them, while stem tips and beams stay where round heads would put them; half and whole notes use their own X glyphs.

3.6 Harmony symbols

Use a bar's `harmony` field for chord symbols above the top staff. It is an independent duration-bearing sequence: each token is author-controlled text followed by `:` and a duration, and the sequence must fill the active bar. Harmony changes use the same onset positions as the notes, including changes that fall between melody note onsets. Each symbol is centered on the onset where the harmony becomes active, independent of its duration. Symbols are text, so jazz spellings such as `Cmaj7` , `F#7(b9)` , `Bb/D` , and `N.C.` remain under the author's control.

```typst
#score(
  time: "4/4",
  bars: (
    (
      notes: "c5:q d5:q e5:q f5:q",
      harmony: "Cmaj7:h A7:h",
    ),
    (
      notes: "g5:h e5:h",
      harmony: "Dm7:q G7:q Cmaj7:h",
    ),
  ),
)
```

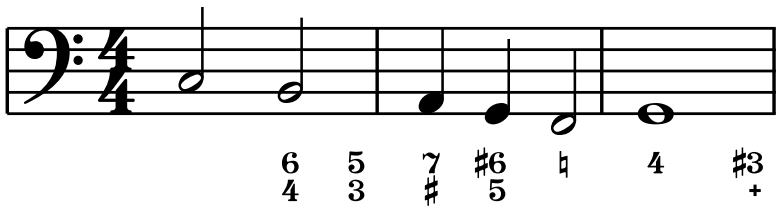


3.7 Figured bass

Use a bar's `figures` field for continuo figures below the bottom staff. Like harmony, it is a duration-bearing sequence that must fill the active bar, and each token names what applies from its onset. A token is one figure such as `6:q` , a stack such as `(6 4):q` written top to bottom, or `_:q` for a stretch without figures. Figures may change during a held bass note; each change gets its own onset column.

Each figure is a number with an optional accidental (`#` , `b` , `n` , `##` , or `bb`) or `+` before or after it, printed where it is written: `#6` gives ♯6 and `6b` gives 6♭. An accidental alone, such as `#` or `n` , alters the third above the bass, and `_` inside a stack keeps an empty row. Figures are drawn with Bravura's figured-bass glyphs, the number centered on the bass note and its accidentals hanging outside. They sit in one band per system under the bottom staff's notes and dynamics; lyrics sung on that staff move below the band.

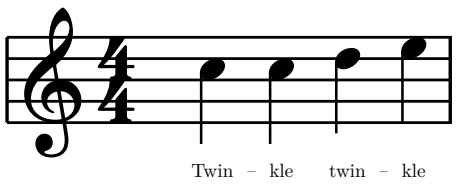
```typst
#score(
  clef: "bass",
  time: "4/4",
  bars: (
    (notes: "c3:h b2", figures: "_:h (6 4):q (5 3):q"),
    (notes: "a2:q g2 f2:h", figures: "(7 #):q (#6 5):q n:h"),
    (notes: "g2:w", figures: "(4 _):h (#3 +):h"),
  ),
)
```



3.8 Lyrics

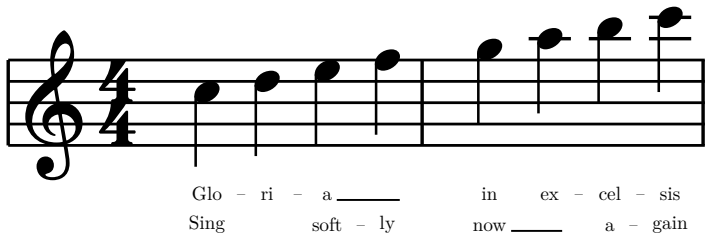
Use a bar's `lyrics` field to align syllables with the main pitched events of a staff's first voice. A plain string creates one verse. `--` draws a hyphen without consuming an event, `__` consumes one event and extends the preceding syllable, and `_` consumes one event without printing text. Written rests and grace notes do not consume lyric tokens. Every verse present in a bar must account for all of that bar's main pitched events, which keeps alignment mistakes visible instead of silently shifting later words.

```typst
#score(
  time: "4/4",
  bars: (
    (
      notes: "c5:q c d e",
      lyrics: "Twin -- kle twin -- kle",
    ),
  ),
)
```



An array creates multiple verses. Use an underscore inside a syllable token to print a space, while a standalone underscore remains a skip. Hyphens and melisma extenders may cross barlines and resume when the score wraps.

```typst
#score(
  time: "4/4",
  width: 34,
  bars: (
    (
      notes: "c5:q d e f",
      lyrics: ("Glo -- ri -- a __", "Sing _ soft -- ly"),
    ),
    (
      notes: "g5:q a b c6",
      lyrics: ("in ex -- cel -- sis", "now __ a -- gain"),
    ),
  ),
)
```



In a declared multi-staff score, `lyrics` is a dictionary whose keys are staff IDs. Each value is one verse string or an array of verses. Omit staves that have no lyrics in that bar. Lyrics always follow the first voice of their target staff; use that voice for the sung line when a staff is polyphonic.
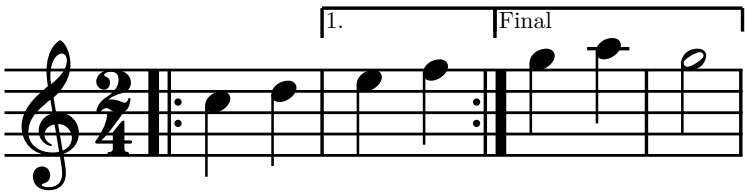
```typst
lyrics: (
  soprano: ("Hal -- le -- lu -- jah", "Praise _ the Lord"),
  bass: "Low __",
)
```

Lyric widths constrain the shared onset grid before measures are packed, so long syllables can widen bars and change line breaks. Verse lanes are also part of the staff's vertical ink extent: they clear dynamics and other below-staff marks, enlarge automatic multi-staff gaps, and contribute to system bounds. Use `lyric-size` , `lyric-font` , `lyric-gap` , and `verse-gap` to tune their typography and vertical rhythm.

3.9 Barlines, navigation, and labeled endings

Repeat signs belong to a bar edge. Use `repeat-start` on a left edge and `repeat-end` on a right edge. A labeled ending is a span: mark its first bar with `start: true` and its last bar with `stop: true`. `label` is rendered literally, so punctuation and text such as `"1."` or `"Final"` are both valid.

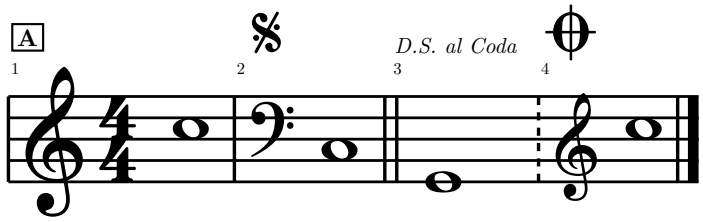
```typst
#score(
  time: "2/4",
  bars: (
    (barline: (left: "repeat-start"), notes: "c5:q d5:q"),
    (
      barline: (right: "repeat-end"),
      ending: (label: "1.", start: true, stop: true),
      notes: "e5:q f5:q",
    ),
    (ending: (label: "Final", start: true), notes: "g5:q a5:q"),
    (ending: (label: "Final", stop: true), notes: "g5:h"),
  ),
)
```



A repeat end followed immediately by a repeat start becomes one double-sided repeat barline. Volta brackets continue across system wraps. Endings are currently sequential: one bracket must stop before another one starts.

A right bar edge may also be `double`, `final`, or `dashed`. Use a bar's `rehearsal` field for a boxed rehearsal label and `navigation` for `"segno"`, `"coda"`, arbitrary text, or Typst content. Marks attach to the beginning of their bar. When a bar number and mark coincide, the number stays closest to the staff and the mark stacks above it. Long boundary text reserves enough horizontal space to clear the following bar. `bar-numbers: "systems"` prints the first bar number of later systems, matching LilyPond's usual sparse numbering; `"all"` prints every bar number.

```typst
#score(
  time: "4/4",
  bar-numbers: "all",
  bars: (
    (rehearsal: "A", notes: "c5:w"),
    (clef: "bass", navigation: "segno", barline: (right: "double"), notes: "c3:w"),
    (navigation: "D.S. al Coda", barline: (right: "dashed"), notes: "g2:w"),
    (clef: "treble", navigation: "coda", barline: (right: "final"), notes: "c5:w"),
  ),
)
```



3.10 Score arguments

Argument	Type	Default	Description
`clef`	`str`	`"treble"`	Clef for the implicit single staff only.
`staves`	`dictionary | none`	`none`	Permanent staff-ID map. Each spec has `clef`, plus optional first-system `label` and later-system `short-label`. A `tab` staff also accepts `tuning` and `source` (Section 5).
`bars`	`array`	`()`	Required non-empty array of bar dictionaries.
`key`	`str`	`"C"`	Initial key signature.
`time`	`str`	`"4/4"`	Initial meter and full-bar duration.
`transpose`	`str | none`	`none`	Interval from the pitches as typed to the pitches as drawn, such as `M6` or `-M2`. Moves notes, key signatures, and chord symbols.
`theme`	`str`	`"auto"`	Notation ink: `light` uses black, `dark` uses white, and `auto` follows the surrounding solid text fill.
`tempo`	`str | content | dictionary | none`	`none`	Tempo text, or `(text:, beat:, bpm:)` for an engraved metronome mark.
`composer`	`str | content | none`	`none`	Composer credit above the first system.
`width`	`number | none`	`none`	Packing width in staff spaces.
`scale`	`number`	`1.0`	Uniform staff-space scale.
`note-spacing`	`number`	`3.1`	Horizontal density: the width in staff spaces of a quarter note in a bar of quarters; other durations scale logarithmically from the bar's shortest note.
`lyric-size`	`number`	`0.9`	Lyric text size in staff spaces.
`lyric-font`	`str | none`	`none`	Optional lyric font family.
`lyric-gap`	`number`	`0.8`	Clearance between existing below-staff ink and the first verse.
`verse-gap`	`number`	`1.45`	Baseline distance between lyric verses; must be at least `lyric-size`.
`beams`	`bool`	`false`	Connect automatic beam groups.
`chord-diagrams`	`dictionary | none`	`none`	Harmony symbols to fretboard shapes, such as `guitar-chords` draws a diagram above each symbol (Section 5.3).
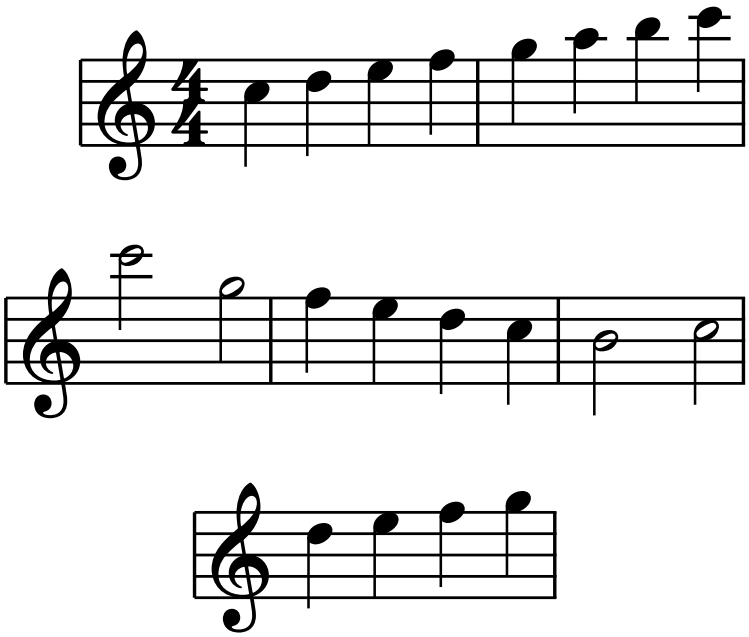
staff-gap	number | none	none	Fixed distance between adjacent staff lines.
group	auto | str	auto	Auto, brace, bracket, line, or none. Auto brackets a score with a tab staff.
wrap	bool	true	Wrap complete bars into systems.
indent	number	0	First-system indentation in staff spaces. The system still ends at the normal right edge.
short-indent	number	0	Indentation for every system after the first.
ragged-right	auto | bool	auto	Leave every system at natural width within the wrapping width. Auto is ragged only for a one-system score.
ragged-last	bool	false	Leave only the final system at natural width within the wrapping width.
system-gap	length	1.2em	Vertical gap between systems.
bar-numbers	false | str	false	Use `systems` for later system starts or `all` for every bar.
first-bar-number	int	1	Positive number assigned to the first bar.

3.11 Indentation and justification

Multi-system scores fill `width` by default: note spacing stretches or compresses so each system's bars fill the line, and adjacent systems keep a similar density rather than alternating cramped and loose lines. There is no fixed bar count per line. A one-system score stays at natural width with the default `ragged-right: auto` , bounded by the wrapping width. Clefs and keys repeat at each system start; a time signature appears only where the meter is first set or changes.

Use `indent` for the first system and `short-indent` for every later one; both reduce the usable line width while retaining a shared right edge. Set `ragged-last: true` for a natural-width final line.

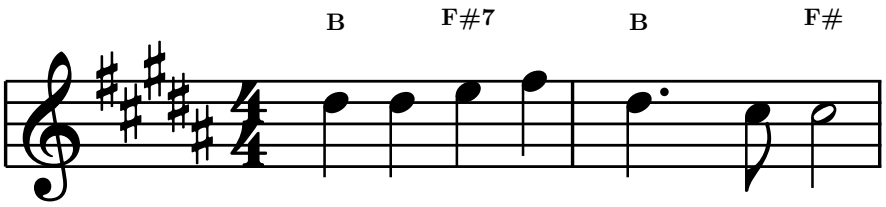
```typst
#score(
  time: "4/4",
  width: 36,
  indent: 3.5,
  ragged-last: true,
  bars: (
    (notes: "c5:q d e f"),
    (notes: "g5:q a b c6"),
    (notes: "c6:h g5:h"),
    (notes: "f5:q e d c"),
    (notes: "b4:h c5:h"),
    (notes: "d5:q e f g"),
  ),
)
```



3.12 Transposition and parts

Write music at concert pitch and let `transpose` produce a transposing instrument's written part. The value is an interval from the pitches as typed to the pitches as drawn: a quality (`P`, `M`, `m`, `A`, or `d`) and a size, with a leading `-` for downward. Every note, key signature, and chord-symbol root moves by that interval. Relative octaves resolve as typed, before the interval applies, so the event strings read exactly as they would untransposed.

```typst
#score(
  key: "D",
  time: "4/4",
  transpose: "M6",
  bars: (
    (notes: "F#4:q F# G A", harmony: "D:h A7:h"),
    (notes: "F#:q. E:e E:h", harmony: "D:h A:h"),
  ),
)
```
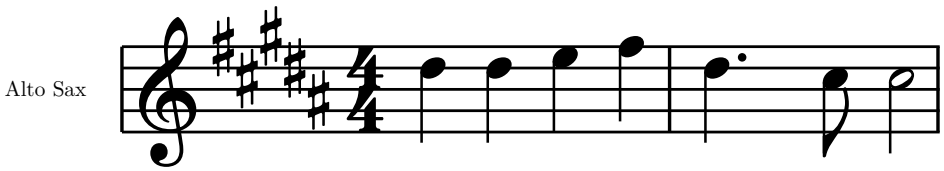


Instrument	Written part from concert pitch
B-flat clarinet, trumpet, soprano saxophone	`transpose: "M2"`
E-flat alto saxophone	`transpose: "M6"`
B-flat tenor saxophone, bass clarinet	`transpose: "M9"`
E-flat baritone saxophone	`transpose: "M13"`
F horn, English horn	`transpose: "P5"`
Guitar, double bass	`transpose: "P8"`
Piccolo	`transpose: "-P8"`

A key keeps the interval's spelling unless that would need more than six sharps or flats (or more than the typed key already has). Then the part takes the enharmonic key, and its notes and chord symbols are spelled to match: concert F-sharp major for alto saxophone is written in E-flat major rather than D-sharp major. A note that would need a triple sharp or flat is respelled on a neighboring letter.

Percussion pitches and their drum-map entries retain their original positions during transposition.

To write a part from an ensemble score, keep the score's arguments in a dictionary and pass them through `part` with a declared staff ID. It returns arguments for a one-staff score: the chosen staff's content, clef changes, and lyrics, with every bar's key, meter, tempo, harmony, barlines, endings, and marks. The ensemble-only `staff-gap` and `group` are dropped. A mirrored tab staff cannot be extracted without its source: extract the notation staff or supply independent tab content. The same dictionary engraves the concert score with `score(..duet)` , and the extracted part spreads into `score` beside any argument for the part itself:

```typst
#let duet = (
  staves: (
    sax: (clef: "treble", label: "Alto Sax"),
    cello: (clef: "bass", label: "Cello"),
  ),
  key: "D",
  time: "4/4",
  bars: (
    (sax: "F#4:q F# G A", cello: "D3:w"),
    (sax: "F#:q. E:e E:h", cello: "A2:w"),
  ),
)
#score(..part(duet, "sax"), transpose: "M6")
```



A staff whose voices use `@staff` switches or split chords depends on its neighbors and cannot be extracted on its own.

4 Event language

Syntax	Meaning
`c4:q` or `c4q`	Quarter note.
`ce` or `c:e`	Relative C with an eighth-note duration.
`c4:q d e f`	Four quarter notes using inherited register and duration.
`bb4:e.`	Dotted eighth note.
`c##5:q` / `dbb5:q`	Double-sharp / double-flat quarter note.
`(a4 c e):h (f a c)`	Relative chord pitches and inherited duration.
`r:q`	Quarter rest.
`s:q`	Invisible quarter spacer: takes time, draws nothing.
`_`	Automatic rest placeholder.
`c2:e g2 @upper e4 g4`	Draw the following events on staff `upper` .
`(c3 g3 @upper e4 c5):h`	Split chord: one stem across two staves.
`~`	Tie the previous event to the next event.
`/`	Break the automatic beam before the next event.
`-`	Join adjacent flagged events into one beam group.
`tuplet 3:2 { c:e d e }`	Inline time-scaled music group.
`tuplet 3:2[number=never] { c:e d e }`	Tuplet without its numeral.
`cue { c:e d }`	Cue-sized notes that keep their rhythmic value.
`acciaccatura { d:e } f:q`	Slashed single grace note before F.
`tremolo 16 { c:h g:h }`	Two-note alternating sixteenth tremolo.

4.1 Duration inheritance

Durations are `w`, `h`, `q`, `e`, `s`, and `t`; append `.` or `..` for dots. An explicit duration becomes the current duration for that staff. Subsequent notes, chords, and ordinary rests may omit it and inherit the complete value, including dots, across barlines. With no preceding value, the default is `q`. The computed-rest placeholder `_` does not change this state, and a written colon must always be followed by a duration. An omitted duration never expands to fill the measure; normal bar-duration validation still reports an incorrect total.

4.2 Pitch spelling and relative octaves

Pitch letters are ASCII case-insensitive. `c4`, `C4`, and mixed-case input have identical pitch and octave semantics; this guide uses lowercase as the compact style, while uppercase remains valid. Compact `ce` and explicit `c:e` both mean an eighth-note C. Whitespace distinguishes that single event from `c e`, which means two notes using the inherited duration. Capitalization never selects an octave.

Pitches accept `#`, `b`, `##`, or `bb`. In lowercase, `bb4` is B-flat 4 and `bbb4` is B-double-flat 4. A single note with an explicit octave anchors later octave-less single notes. Each omitted octave selects the nearest diatonic pitch to the preceding resolved note, so `g4:e a b c` ends on C5 and all four notes are eighths. Accidentals and the active key signature do not affect that register choice. The anchor continues across

bars independently for each staff, and any explicit octave resets it. At the beginning of an unanchored staff, treble, alto, tenor, and percussion start in octave 4; bass starts in octave 3.
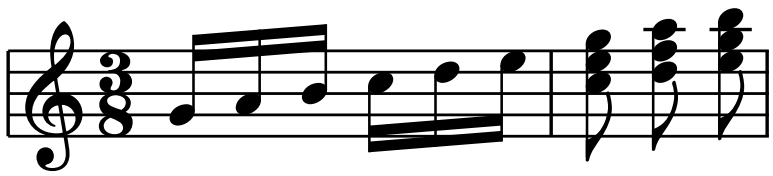
4.3 Relative chords

Chords follow LilyPond's relative-entry model. The first written chord pitch is resolved from the current staff anchor. Each later chord pitch is resolved from the pitch immediately before it inside the chord. Afterward, the first written chord pitch becomes the external anchor for the next event. For example, `(a4 c e):h (f a c)` resolves to `(A4 C5 E5):h (F4 A4 C5):h`. Explicit octaves remain exact overrides for open or otherwise non-nearest voicings.

4.4 Beam boundaries

Standalone `/` and `-` are local beam-boundary controls. `/` splits before the next event; `-` joins the flagged event on each side even across an automatic metric boundary. A join at an edge, beside a rest, or beside an unflagged duration is an error. For example, this 3/8 measure is grouped 3+3 rather than the automatic 2+2+2 sixteenths, and the next bar demonstrates relative chord voicing. No `key` argument is needed here: `score` defaults to C major.

```typst
#score(
  time: "3/8",
  beams: true,
  bars: (
    (notes: "g4:s a - b / c - d e"),
    (notes: "(c5 e g):e (d f a) (e g b)"),
  ),
)
```

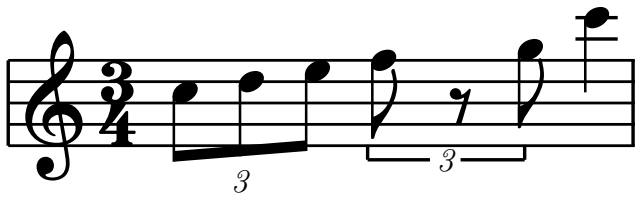


4.5 Tuplets

Tuplets are inline music groups: `tuplet 3:2 { c:e d e }` writes three eighths in the time normally occupied by two. Any positive ratio is supported, and groups may nest. The group preserves pitch and duration inheritance, so only its first event needs an explicit octave or duration when the surrounding context already supplies one.

By default the numerator is centered on the group, and a bracket is omitted when one visible beam spans every event in the tuplet; otherwise a bracket is drawn. Written rests keep the bracket. Use optional group controls when the engraving needs an explicit choice: `bracket=always`, `bracket=never`, `side=above`, or `side=below`. `number=never` hides the numeral independently of the bracket, and `number=always` restores the default; combine `number=never` with `bracket=never` to hide the tuplet entirely. Each option may appear once, and unknown values are reported.
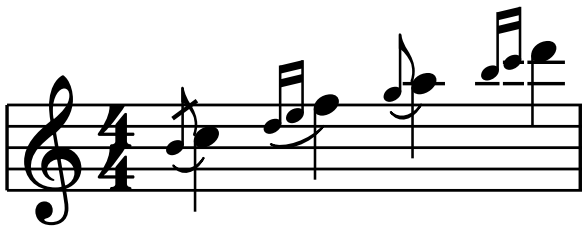
```typst
#score(
  time: "3/4",
  beams: true,
  bars: (
    (
      notes: "tuplet 3:2 { c5:e d e } tuplet 3:2 { f5:e r:e g5:e } c6:q",
    ),
  ),
)
```



4.6 Grace notes

Grace groups stay inline and consume no bar time. `grace { ... }` writes plain small grace notes. `acciaccatura { ... }` adds a slur to the following principal note, as does `appoggiatura { ... }`. For a multi-note group, the slur starts at the first grace event. A single flagged acciaccatura also receives the conventional slash; a multi-note beamed acciaccatura does not. Consecutive flagged grace notes beam as an independent group. Their written durations control flags and beams but do not contribute to the measure total.

```typst
#score(
  time: "4/4",
  beams: true,
  bars: (
    (notes: "acciaccatura { b4:e } c5:q acciaccatura { d5:s e } f5:q appoggiatura
    { g5:e } a5:q grace { b5:s c6 } d6:q"),
  ),
)
```



A grace group must contain notes or chords and must precede a principal event. Rests, automatic rests, tuplets, alternating tremolos, and nested grace groups are rejected inside it so timing remains unambiguous.

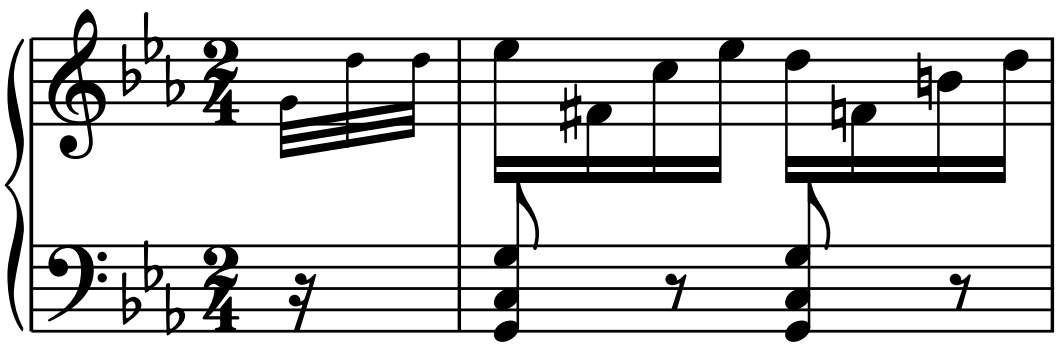
4.7 Cue-sized notes

`cue { ... }` draws notes, chords, and written rests at the reduced grace-note size while keeping their full rhythmic value. Unlike grace notes, they count toward the measure total and align with other staves like normal events. Use a cue group for a small but measured figure, such as a tuplet upbeat. Cue groups may contain tuplets, and tuplets may contain cue groups. A tuplet whose events are all cue-sized also draws a smaller numeral and bracket.

Cue-sized and normal-sized notes never share a beam: an automatic beam breaks at the group boundary, and a `-` join across it is an error. Fingerings, articulations, dynamics, and other annotations keep their normal size.

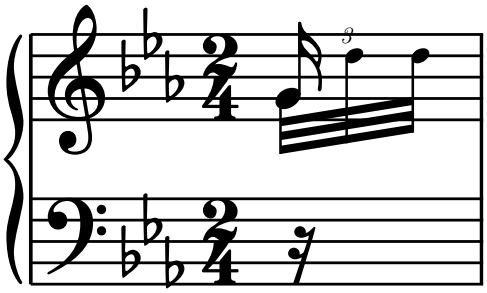
Typst

```
#score(
  staves: (
    upper: (clef: "treble"),
    lower: (clef: "bass"),
  ),
  key: "Cm",
  time: "2/4",
  beams: true,
  bars: (
    (
      partial: "1/16",
      upper: "cue { tuplet 3:2[number=never] { g4:t d5 d } }",
      lower: "r:s",
    ),
    (
      upper: "eb5:s f#4 c5 eb d f4 b d",
      lower: "(g2 c3 g3):e r (g2 c3 g3):e r",
    ),
  ),
)
```



Combined with independent voices, a cue group can share a measured upbeat with a normal-sized note. Here the normal G stems upward in voice 1, while the cue-sized triplet stems downward in voice 2 and merges with the coincident G notehead. `side=above` keeps the numeral on the notehead side, clear of the beams.

```typst
#score(
  staves: (
    upper: (clef: "treble"),
    lower: (clef: "bass"),
  ),
  key: "Cm",
  time: "2/4",
  beams: true,
  bars: (
    (
      partial: "1/16",
      upper: ("g4:s", "cue { tuplet 3:2[side=above] { g4:t d5 d } }"),
      lower: "r:s",
    ),
  ),
)
```



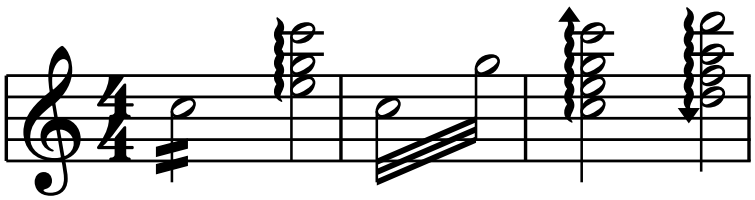
A cue group must contain at least one note, chord, or written rest. Nested cue groups, grace groups, alternating tremolos, and automatic rests are rejected inside it, as is a beam marker before the group or at its end.

4.8 Tremolos and arpeggios

For a repeated single event, append `tremolo=8`, `16`, `32`, or `64` in its annotation block. These values draw one, two, three, or four strokes respectively, independent of the event's written duration. For an alternation, place exactly two equal-duration notes or chords in `tremolo subdivision { ... }`; the connecting strokes replace ordinary flags or beams.

Append `arpeggio`, `arpeggio=up`, or `arpeggio=down` to a chord. The arpeggio wave spans the full chord immediately to its left; the directional forms add an arrow at the appropriate end.

```typst
#score(
  time: "4/4",
  bars: (
    (notes: "c5:h[tremolo=16] (e5 g5 c6):h[arpeggio]"),
    (notes: "tremolo 16 { c5:h g5:h }"),
    (notes: "(c5 e5 g5 c6):h[arpeggio=up] (d5 f5 a5 d6):h[arpeggio=down]"),
  ),
)
```



4.9 Expressive annotations

Annotations follow an event in square brackets. Supported annotations include fingering (`f=4`), staccato, staccatissimo, tenuto, accent, marcato, text directions, ornaments (turns, trills, and mordents), fermatas (`fermata`), breath marks (`breath`), named slurs, pedal spans, hairpins, ottava brackets, and dynamics (`dyn=p`, `dyn=pp`, `dyn=mf`, `dyn=sfz`, and the other standard SMuFL combinations).

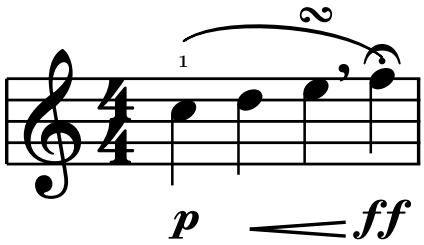
Every ornament occupies one script slot above the note, so an event may carry at most one of them. The turn family — `turn`, its reverse `inverted-turn`, and the accidental-bearing `chromatic-turn` — accepts a fingered digit through `turn-f=`.

Dynamics, hairpins, slurs, articulations, and fingerings avoid collisions with each other automatically: a hairpin shortens around an adjacent dynamic such as a closing `dyn=ff`, and fingerings rise above stacked articulations rather than overlapping them. Named slurs over beamed passages follow the beam group's stem direction, so a slur goes below when every note it spans stems upward and above when any note stems downward.

Syntax	Mark
`f=4`	Fingering.
`stacc` / `staccatissimo`	Staccato dot / wedge.
`tenuto` / `accent` / `marcato`	Articulation marks.
`turn` / `inverted-turn` / `chromatic-turn`	Turn ornaments.
`trill`	Trill (*tr*).
`mordent` / `inverted-mordent`	Lower / upper mordent.
`dyn=pp` / `dyn=sfz`	Dynamic marking.
`fermata` / `breath`	Fermata / comma breath mark.
`s1(` … `s1)`	Named slur.
`p1(` … `p1)`	Pedal span.
`h1<` or `h1>` … `h1!`	Crescendo / diminuendo hairpin.

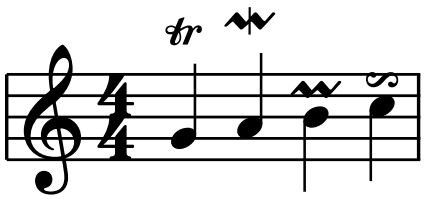
`x` / `circle-x` / `normal`	Notehead shape, overriding any drum map; also after one chord pitch.
`8va(` … `8va)`	Ottava bracket; also `8vb` , `15ma` , and `15mb` .

```typst
#score(
  clef: "treble",
  time: "4/4",
  beams: true,
  bars: (
    (notes: "c5:q[f=1 s1( dyn=p] d5:q[h1<] e5:q[turn breath] f5:q[s1) h1! dyn=ff fermata]"),
  ),
)
```



Ornaments share the turn's placement above the note, so trills and mordents stack over articulations and under fingerings just as a turn does.

```typst
#score(
  clef: "treble",
  time: "4/4",
  bars: (
    (notes: "g4:q[trill] a4:q[mordent] b4:q[inverted-mordent] c5:q[inverted-turn]"),
  ),
)
```

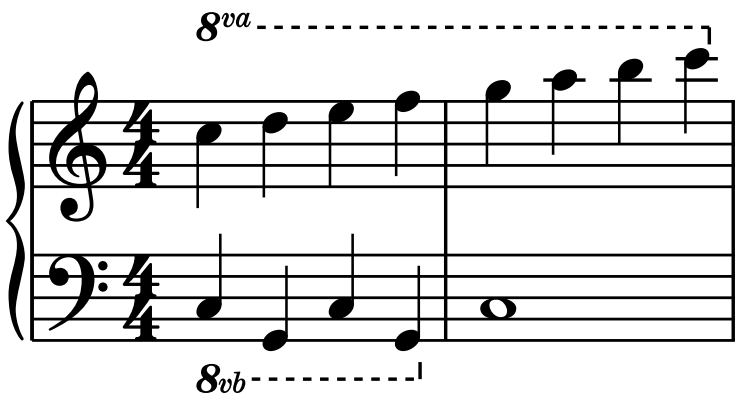


4.10 Ottava brackets

An ottava span opens with `8va(` and closes with `8va)` on the event that ends it, which may be the same event. Write the notes at the pitch they sound, as in LilyPond: under `8va` they are drawn an octave lower, under `15ma` two octaves lower, and `8vb` and `15mb` draw them one or two octaves higher. Relative octaves, ties, and chords therefore keep working on the sounding pitches, and accidentals hold for the line or space where a note is drawn.

A dashed bracket names the shift, above the staff for `8va` and `15ma` and below it for `8vb` and `15mb` , and ends in a hook at the last note. It spans barlines and continues across system breaks, where the continuation shows the bare numeral. Brackets clear the notes, articulations, and slurs beneath them; chord names, voltas, and tempo marks rise above an ottava on the top staff, and a bracket below the staff hangs under the voice's dynamics. A span over a single note shows the sign alone. Each voice holds one ottava at a time, and the events of a span stay on the voice's own staff.

```typst
#score(
  staves: (upper: (clef: "treble"), lower: (clef: "bass")),
  time: "4/4",
  bars: (
    (
      upper: "c6:q[8va(] d e f",
      lower: "c2:q[8vb(] g1 c2 g1[8vb)]",
    ),
    (
      upper: "g6:q a b c7[8va)]",
      lower: "c3:w",
    ),
  ),
)
```



4.11 Cross-staff notation

Piano music often splits one gesture between the staves: an arpeggio climbs from the bass into the treble, a run crosses the hands, or a chord's lower notes sit on the bass staff while its upper notes sit on the treble. In typed-scores a voice always belongs to its home staff, the staff whose field holds its text, and that is where its rhythm, beams, slurs, dynamics, pedals, and lyrics live. Individual events can be drawn on another staff without leaving the voice.

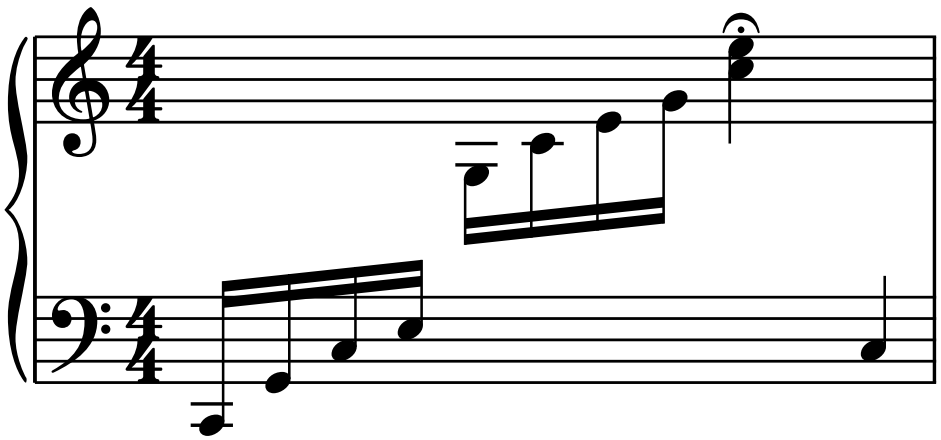
Staff switches

`@staff` draws every following event of the voice on the named staff, using that staff's clef. The switch holds until the next `@staff` and resets to the home staff at every bar, so each bar's text reads on its own. Write the switch as its own word, separated by spaces. Switches may appear inside tuplets and cue groups. They may also come just before a grace group, but not inside one or inside an alternating tremolo.

Relative octaves ignore the switch: `@lower g` after `c4` still resolves to the G nearest C4. A note drawn away from its home staff points its stem back toward that staff, so the hand playing it stays visible.

Use the spacer `s` where a staff has no notes of its own. It takes time like a rest but draws nothing, and it accepts the same durations, including inherited ones.

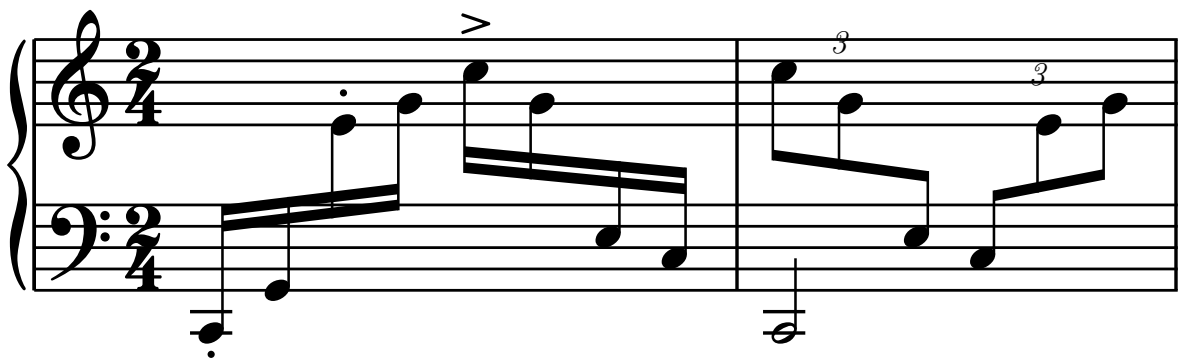
```typst
#score(
  staves: (
    upper: (clef: "treble"),
    lower: (clef: "bass"),
  ),
  time: "4/4",
  beams: true,
  bars: (
    (
      upper: "s:h e5:q[fermata] s",
      lower: "c2:s g2 c3 e3 @upper g3 c4 e4 g4 c5:q @lower c3",
    ),
  ),
)
```



Kneed beams

When one automatic beam group holds notes from two neighboring staves, the beam is kneed: it runs between the staves, stems from the upper staff point down to it, and stems from the lower staff point up. The gap between the staves grows until each stem keeps a comfortable length. Secondary beams stack toward the notes where the group begins. Articulations follow the notehead side of each stem. A voice with forced stem directions keeps them and draws an ordinary beam beside its notes.
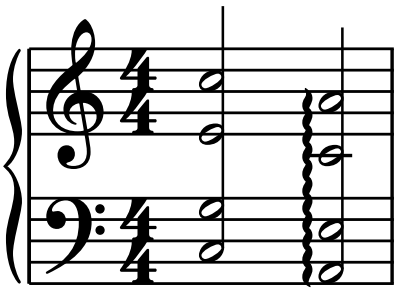
```typst
#score(
  staves: (
    upper: (clef: "treble"),
    lower: (clef: "bass"),
  ),
  time: "2/4",
  beams: true,
  bars: (
    (
      upper: "s:h",
      lower: "c2:s[stacc] g2 @upper e4[stacc] g4 c5[accent] g4 @lower e3 c3",
    ),
    (
      upper: "tuplet 3:2 { c5:e g4 @lower e3 } tuplet 3:2 { c3 @upper e4 g }",
      lower: "c2:h",
    ),
  ),
)
```



Split chords

Inside a chord, `@staff` moves the remaining pitches to a neighboring staff, and one stem joins both parts across the gap. The upper staff must hold the chord's upper register. An `arpeggio` sign on a split chord spans both staves.

```typst
#score(
  staves: (
    upper: (clef: "treble"),
    lower: (clef: "bass"),
  ),
  time: "4/4",
  bars: (
    (
      upper: "s:w",
      lower: "(c3 g3 @upper e4 c5):h (a2 e3 @upper c4 a4):h[arpeggio]",
    ),
  ),
)
```

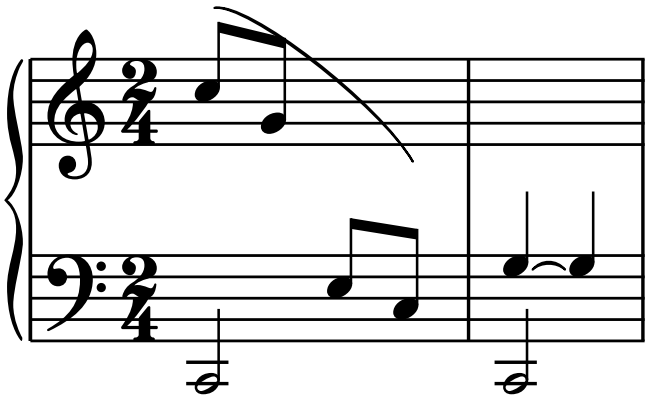


Slurs, ties, and accidentals

Slurs follow the phrase across the staves. A slur whose notes lie on two staves arches above them. A tie needs both notes on the same staff; because switches reset at every bar, a note tied over the barline repeats its `@staff` switch in the next bar. An accidental holds until the end of the bar on the staff that draws it, for every voice drawn on that staff. A note moved to another staff follows that staff's accidentals.

Typst

```
#score(
  staves: (
    upper: (clef: "treble"),
    lower: (clef: "bass"),
  ),
  time: "2/4",
  beams: true,
  bars: (
    (upper: "c5:e[s1(] g4 @lower e3 c[s1)]", lower: "c2:h"),
    (upper: "@lower g3:q ~ g3", lower: "c2:h"),
  ),
)
```
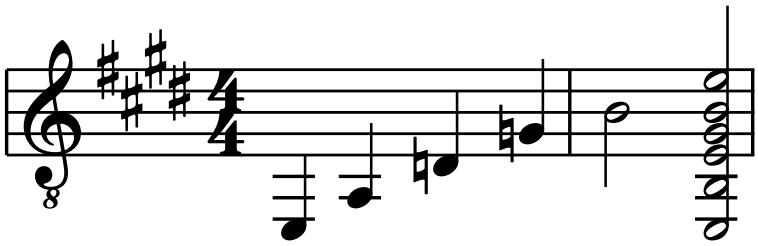


5 Guitar notation

Guitar music is usually printed twice over: standard notation for rhythm and phrasing, and tablature beneath it that tells the player which string and fret to use. typed-scores writes both from one event string. The notation staff uses the guitar's octave clef, a tab staff repeats its music as fret numbers, and chord diagrams can stand above the harmony symbols.

5.1 Guitar clefs

Guitar music is written an octave above its sounding pitch. The `treble-8` clef, a treble clef with a small 8 below, says so on the page: write the pitches the guitar sounds, and each one appears on the line it occupies in guitar music. The open strings are E2, A2, D3, G3, B3, and E4, and the low E sits three ledger lines below the staff. `bass-8` does the same for bass guitar. Both clefs share their plain clef's key signatures, and a first note without an octave starts from octave 3 (`treble-8`) or 2 (`bass-8`).

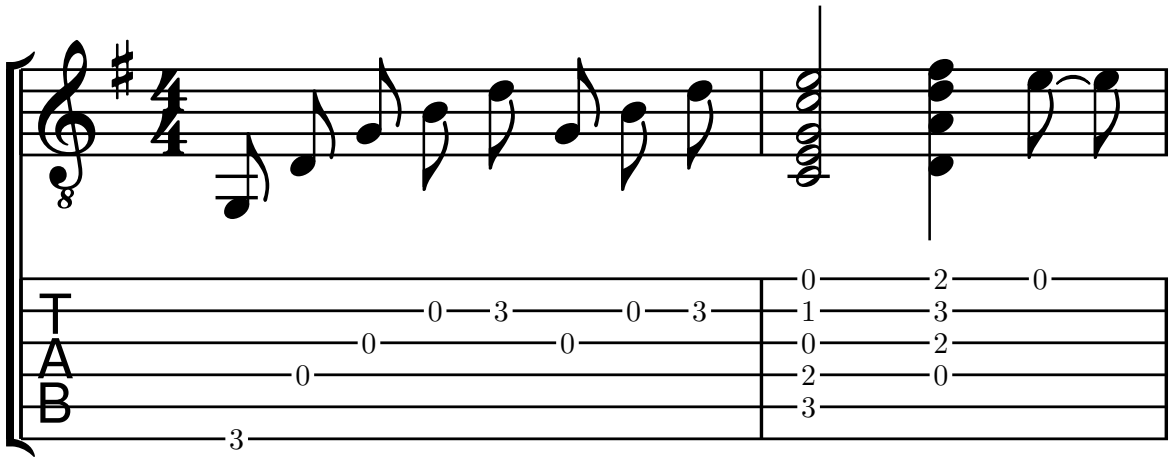
```typst
#score(
  clef: "treble-8",
  key: "E",
  time: "4/4",
  bars: (
    (notes: "e2:q a2 d3 g3"),
    (notes: "b3:h (e2 b2 e3 g#3 b3 e4)"),
  ),
)
```



5.2 Tablature staves

Declare a staff with `clef: "tab"` to draw tablature. Its lines are the strings, the highest string on top, one and a half staff spaces apart, and every note becomes a fret number on its string. `source` names a notation staff whose music the tab repeats, so the bars never write it twice; a bar must not give the tab staff a field of its own. A score with a tab staff groups its staves with a bracket.

```typst
#score(
  staves: (
    guitar: (clef: "treble-8"),
    tab: (clef: "tab", source: "guitar"),
  ),
  key: "G",
  time: "4/4",
  bars: (
    (guitar: "g2:e d3 g3 b3 d4 g3 b3 d4"),
    (guitar: "(c3 e3 g3 c4 e4):h (d3 a3 d4 f#4):q e4:e~ e4"),
  ),
)
```

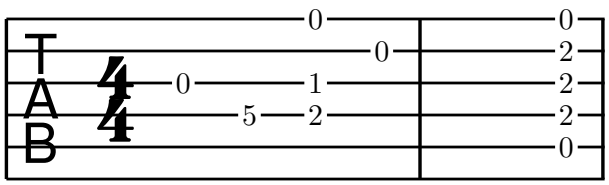


Without `source`, a tab staff reads its own event strings like any staff. The tab reads every pitch as sounding pitch, which is why the guitar staff above uses `treble-8`. A treble staff written at guitar pitch would put every fret an octave too high.

Choosing strings

Each note takes the lowest fret that reaches it, the same default LilyPond uses, so music in open position needs no help. Notes that start together spread across separate strings with the smallest stretch, a note still ringing in another voice keeps its string when it can, and a tied note stays on the string it was tied from. To choose yourself, add `string=` with the string number, where string 1 is the highest; for a chord, list one string per pitch in written order.

```typst
#score(
  clef: "tab",
  time: "4/4",
  bars: (
    (notes: "g3:q g3[string=4] (e3 g#3 e4)[string=4,3,1] b3~"),
    (notes: "b3:h (a2 e3 a3 c#4 e4)"),
  ),
)
```



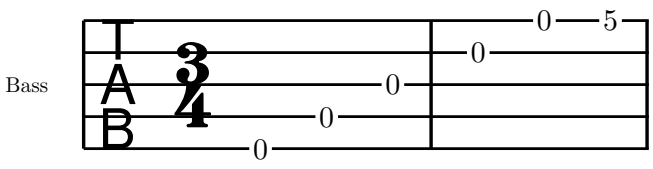
A tab staff without a notation staff beside it prints the time signature, since nothing else shows the meter. Grace and cue notes use smaller numbers. Repeat barlines take their dots in the second and fourth spaces.

Tunings

`tuning` accepts a preset name or an array of open-string pitches, listed from the lowest string to string 1 in the order players name tunings. The number of pitches sets the number of lines. The default is `"guitar"`.

Preset	Open strings, lowest string first
`guitar`	E2 A2 D3 G3 B3 E4
`drop-d`	D2 A2 D3 G3 B3 E4
`dadgad`	D2 A2 D3 G3 A3 D4
`open-g`	D2 G2 D3 G3 B3 D4
`bass`	E1 A1 D2 G2
`ukulele`	G4 C4 E4 A4 (re-entrant)

```typst
#score(
  staves: (
    bass: (
      clef: "tab",
      tuning: ("b0", "e1", "a1", "d2", "g2"),
      label: "Bass",
    ),
  ),
  time: "3/4",
  bars: ((bass: "b0:q e1 a1"), (bass: "d2:q g2 c3")),
)
```



What a tab staff draws

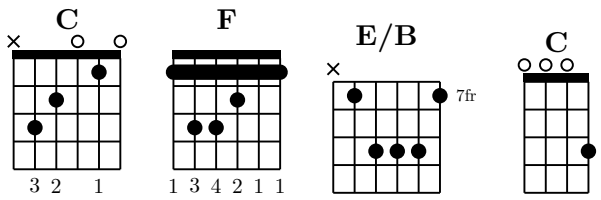
A tab staff draws its clef, fret numbers, and barlines. Rests, stems, beams, tuplet brackets, articulations, dynamics, slurs, and ties stay on the notation staff it repeats. Independent tab accepts string choices and ties but rejects notation annotations it cannot draw; put markings on a notation source staff. A tied note is not restated in the tab, except where it continues onto a new system, where its fret appears in parentheses. Tab lines break around each number instead of being painted over, so the tab reads on any page color. A tab staff keeps its clef for the whole score, and cross-staff switches can neither enter nor leave it.

The parser reports a pitch below the lowest open string or above the 24th fret, a `string=` the pitch cannot reach, and simultaneous notes that cannot share the strings.

5.3 Chord diagrams

`chord-diagram` draws a fretboard box on its own, anywhere in a document. The first argument lists one entry per string from the lowest string: `x` for a muted string, `0` for an open one, or the fret. Write a compact string such as `"x32010"`, separate the entries with spaces or commas once frets reach 10 (`"x 7 9 9 9 7"`), or pass an array such as `(0, 0, 0, 3)`. The number of entries sets the number of strings, so ukulele and bass shapes work the same way.

```typst
#grid(
  columns: 4,
  column-gutter: 1.5em,
  align: bottom,
  chord-diagram("x32010", name: "C", fingers: "032010"),
  chord-diagram("133211", name: "F", fingers: "134211"),
  chord-diagram("x 7 9 9 9 7", name: "E/B"),
  chord-diagram((0, 0, 0, 3), name: "C"),
)
```

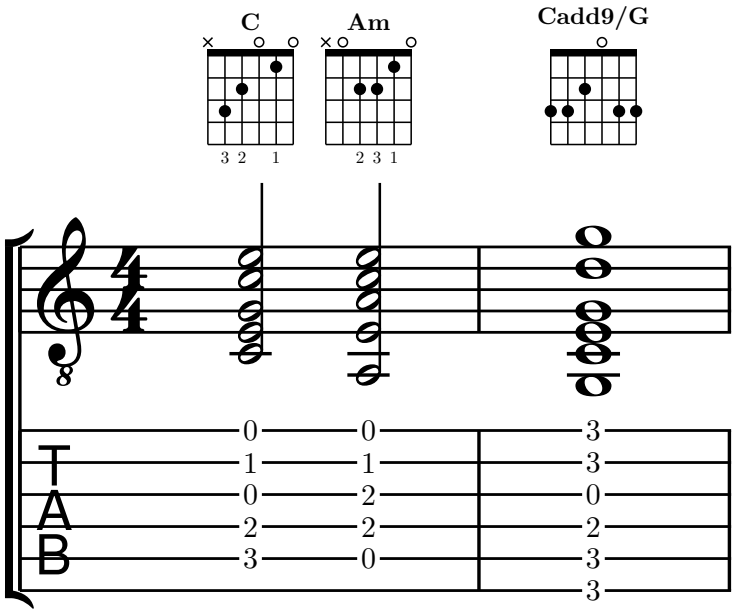


Argument	Type	Default	Description
`frets`	`str | array`	required	One entry per string from the lowest: `x`, `0`, or a fret up to 24.
`name`	`str | content | none`	`none`	Chord name above the diagram.
`fingers`	`str | array | none`	`none`	One finger label per string, printed under the grid; `0`, `x`, or `-` for none.
`barre`	`auto | none | int | array`	`auto`	Auto draws a barre wherever one finger stops several strings at one fret. A fret number or array sets barres explicitly.
`position`	`auto | int`	`auto`	Fret at the top of the grid. Auto keeps the nut for shapes within the first frets and otherwise starts at the lowest fretted note, with a label such as `5fr`.
`fret-count`	`int`	`4`	Minimum number of frets the grid shows.
`scale`	`number`	`1.0`	Uniform size.
`theme`	`str`	`"auto"`	Ink color, as for `score`.

Diagrams above harmony symbols

Pass `chord-diagrams` to `score` to draw a diagram above every harmony symbol. It maps each symbol to a shape: a frets string or array, or a dictionary with `frets` and optional `fingers`, `barre`, `position`, and `fret-count`. `guitar-chords` provides standard-tuning shapes for the major, minor, 7, m7, and maj7 chords on every root, spelled with sharps and with flats, plus Asus2, Asus4, Cadd9, Dsus2, Dsus4, and Esus4. Extend it with `+`. A symbol mapped to `none` has no diagram, and `N.C.` never takes one. Any other symbol missing from the dictionary is reported, so a typo cannot silently drop a diagram.

```typst
#score(
  staves: (
    guitar: (clef: "treble-8"),
    tab: (clef: "tab", source: "guitar"),
  ),
  time: "4/4",
  chord-diagrams: guitar-chords + ("Cadd9/G": "332033"),
  bars: (
    (
      guitar: "(c3 e3 g3 c4 e4):h (a2 e3 a3 c4 e4)",
      harmony: "C:h Am:h",
    ),
    (
      guitar: "(g2 c3 e3 g3 d4 g4):w",
      harmony: "Cadd9/G:w",
    ),
  ),
)
```



The diagrams stand in one row above the top staff's highest ink, with their tops aligned and the chord names above them, and each bar widens as needed to keep neighboring diagrams apart.

6 Importing MusicXML and ABC
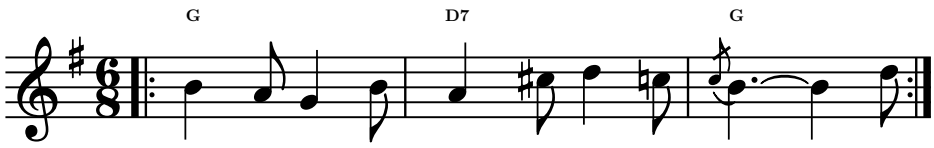
Existing scores do not need to be retyped. `import-score` reads a MusicXML file (`.musicxml`, `.xml`, or compressed `.mxl`, partwise or timewise) or an ABC 2.1 tune and engraves it with the same renderer as hand-written input. Pass the file's contents rather than its path, because a package cannot open files in your project; reading them as bytes works for every format:

```typst
#import "@preview/typed-scores:0.6.0": import-score, read-score

#import-score(read("song.mxl", encoding: none))
#import-score(read("tunes.abc"), tune: 12, scale: 0.8)
```

The format is detected from the contents; pass `format: "musicxml"` or `format: "abc"` to force one, and `tune` to pick an ABC tune by its `X:` number. Other named arguments are `score` options that override the imported ones, such as `scale` , `width` , or `theme` . The work title is centered above the score unless `title` is `none` or other content. An ABC tune can also be written inline:

```typst
#import-score(
  "X:1
T:Morning Air
M:6/8
L:1/8
K:G
|: \"G\"B2A G2B | \"D7\"A2^c d2=c | \"G\"{/c}B3- B2d :|",
  title: none,
)
```



`read-score` returns the conversion instead of engraving it, so the music can be adjusted before rendering. Its `arguments` dictionary holds the `score` arguments (`staves` , `key` , `time` , `tempo` , `composer` , `beams` , and `bars`); `title` is the work title; `warnings` lists what could not be reproduced; and `source` is an equivalent standalone Typst file, for copying into a document and editing as event strings.

```typst
#let song = read-score(read("song.musicxml", encoding: none))
#let bars = song.arguments.bars.slice(0, 8)
#score(..song.arguments, bars: bars, scale: 0.8, bar-numbers: "all")
#raw(song.source, lang: "typ", block: true)
```

Each MusicXML part becomes a labeled staff, and a multi-staff part such as a piano becomes an upper and a lower staff. MusicXML voices become voice strings; a voice that moves to the other staff of its part is written with `@staff` switches. Each ABC voice becomes a staff, and `%%score (1 2)` groups voices onto one staff. Pitches are always written with an explicit octave, and a duration is omitted only when it repeats the previous one. Source beaming is kept by writing `-` and `/` only where it differs from the automatic beams.

Source	Result
Short first or later bar	`partial` with the bar's actual length.
Clef, key, meter, tempo change	Bar metadata; imported mid-bar clefs move to the next barline.
Repeats and endings	`barline` repeat signs and `ending` brackets.
Rehearsal, segno, coda	`rehearsal` and `navigation` fields.
Wedges, pedal lines	Named `h1<` … `h1!` and `p1(` … `p1)` spans.
Chord symbols	`harmony` , with invisible filler symbols where a change falls mid-beat.
Lyrics	Verse strings with `--` , `__` , and `_` tokens.
Hidden rests, voice gaps	Spacers `s` , which draw nothing.

When `read-score` cannot reproduce a construct, its conversion lists the loss: a tie joining only part of a chord or crossing staves is dropped, lyrics under a second voice are dropped, and grace notes inside a tuplet are dropped. Every staff shares the first part's key, so transposing parts lose their own key signature. Each kind of loss is listed in `warnings` and in the header of `source` . `import-score` reports conversion losses as an error. Use `read-score` to inspect the warnings and emitted source before explicitly engraving the reviewed arguments with `score` . Invalid numeric fields, zero durations or denominators, malformed pitches, and more than two dots produce actionable errors. A duration may expand into at most 256 tied note values; divide longer values across shorter measures. Breves, notes shorter than a thirty-second, and a bar holding more music than its meter stop the import with an error naming the bar. Imported scores are written at `scale: 0.7` if a dense bar does not fit the line, lower `scale` or `note-spacing` .

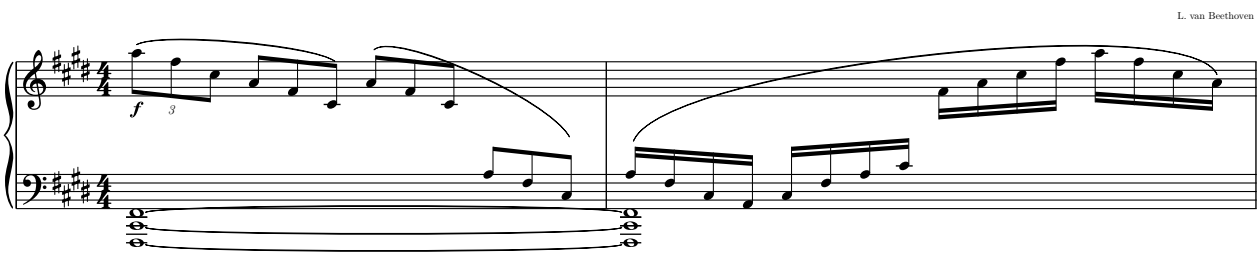
7 Validation and layout

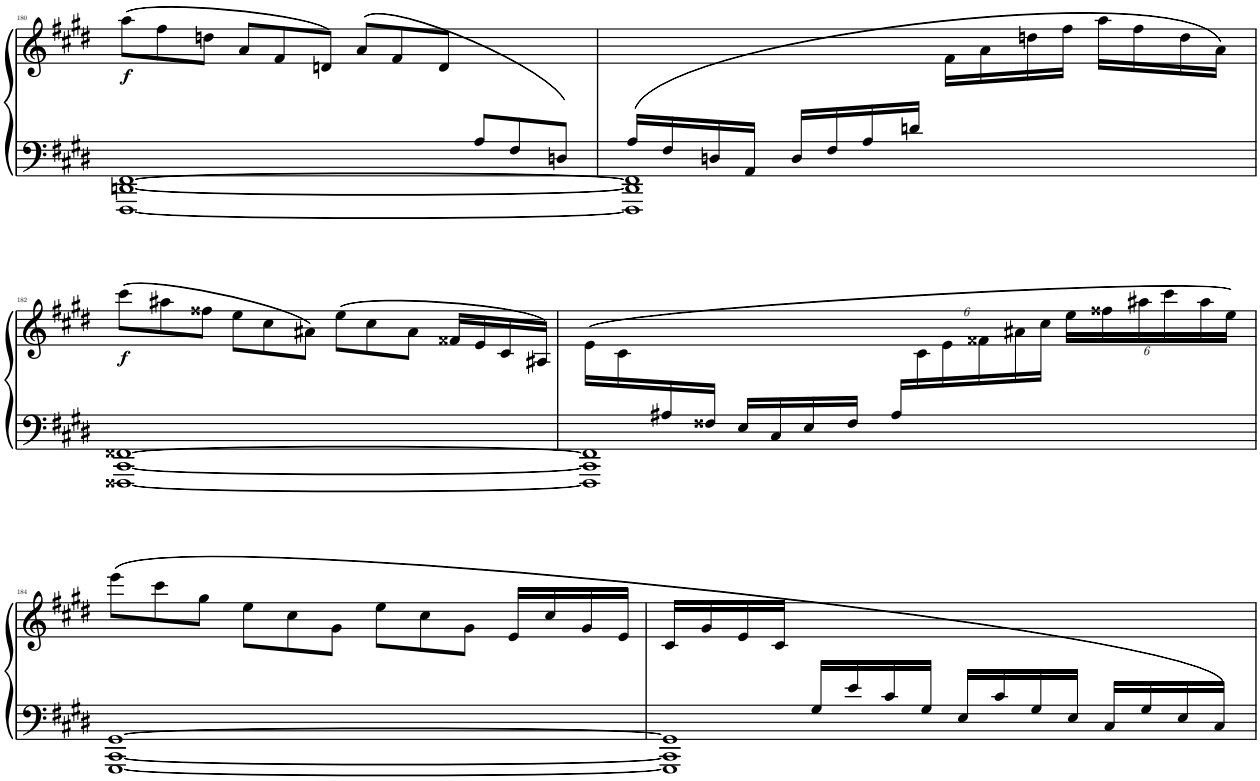
The package validates every full bar against its active `time` signature and every pickup against `partial`. It also reports unknown staff IDs, missing staff content, malformed or undocumented annotations, impossible automatic rests, invalid slur/pedal/hairpin lifecycles, and tie errors with staff and bar context. A tie must connect immediately adjacent events with the same written pitch or chord; it is split cleanly when the score wraps and suppresses a redundant continuation accidental. Multi-staff onsets share horizontal positions, and bars remain atomic when systems wrap.

Condition	Diagnostic
Malformed pitch or duration	The parser identifies the invalid component.
Wrong full-bar duration	Actual reduced duration and expected meter.
Wrong pickup duration	Actual duration and declared `partial` value.
Wrong harmony duration	Actual reduced duration and expected meter.
Invalid lyric token count or control	Bar, staff, verse, expected event count, and a concrete correction.
Unknown lyric staff or dangling continuation	The invalid staff reference, hyphen, or melisma source.
Unknown or missing staff ID	The one-based bar and offending field.
Invalid span lifecycle	The slur, pedal, hairpin, or ending that cannot be paired.
Invalid tie or beam join	The adjacent events that cannot legally connect.
Unknown, redundant, or dangling `@staff` switch	The switch and the staves the score declares.
Split chord out of register or across distant staves	The pitches or staves that one stem cannot join.
Tie across staves or beam across distant staves	The event that must stay on one staff or break its beam.
`staff-gap` too small for a kneed beam	The minimum gap the beam needs.

7.1 Worked example: Beethoven
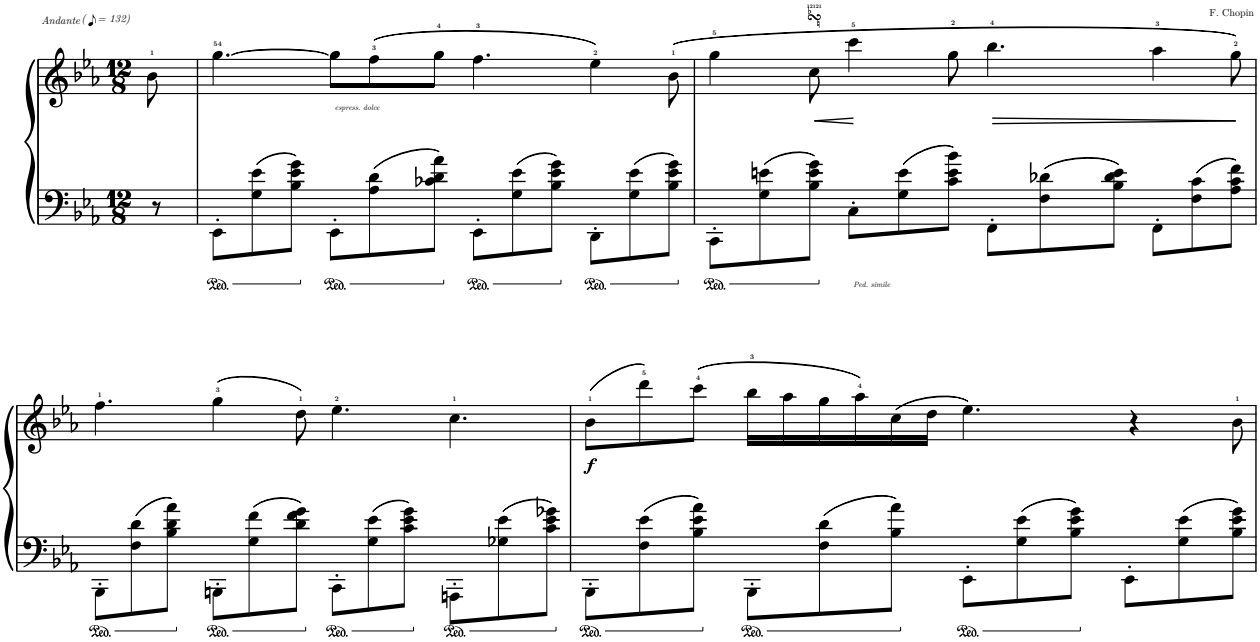
The coda of the “Moonlight” Sonata’s Presto agitato sends the right hand’s arpeggios onto the bass staff and back, with cross-staff slurs and kneed sextuplet beams:
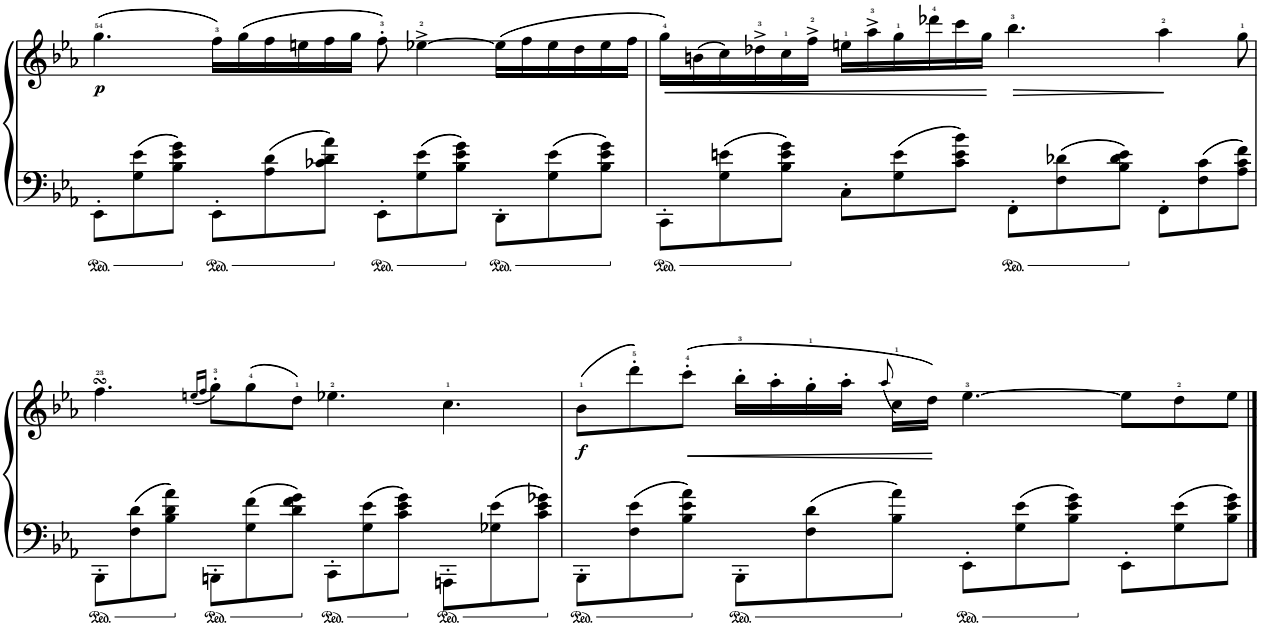




7.2 Worked example: Chopin

The canonical release fixture uses the multi-staff form and exercises a pickup, compound beaming, ties, slurs, dynamics, turns, pedal spans, and chromatic accidentals:




p
Ped.
Ped.
Ped.
Ped.
Ped.
Ped.
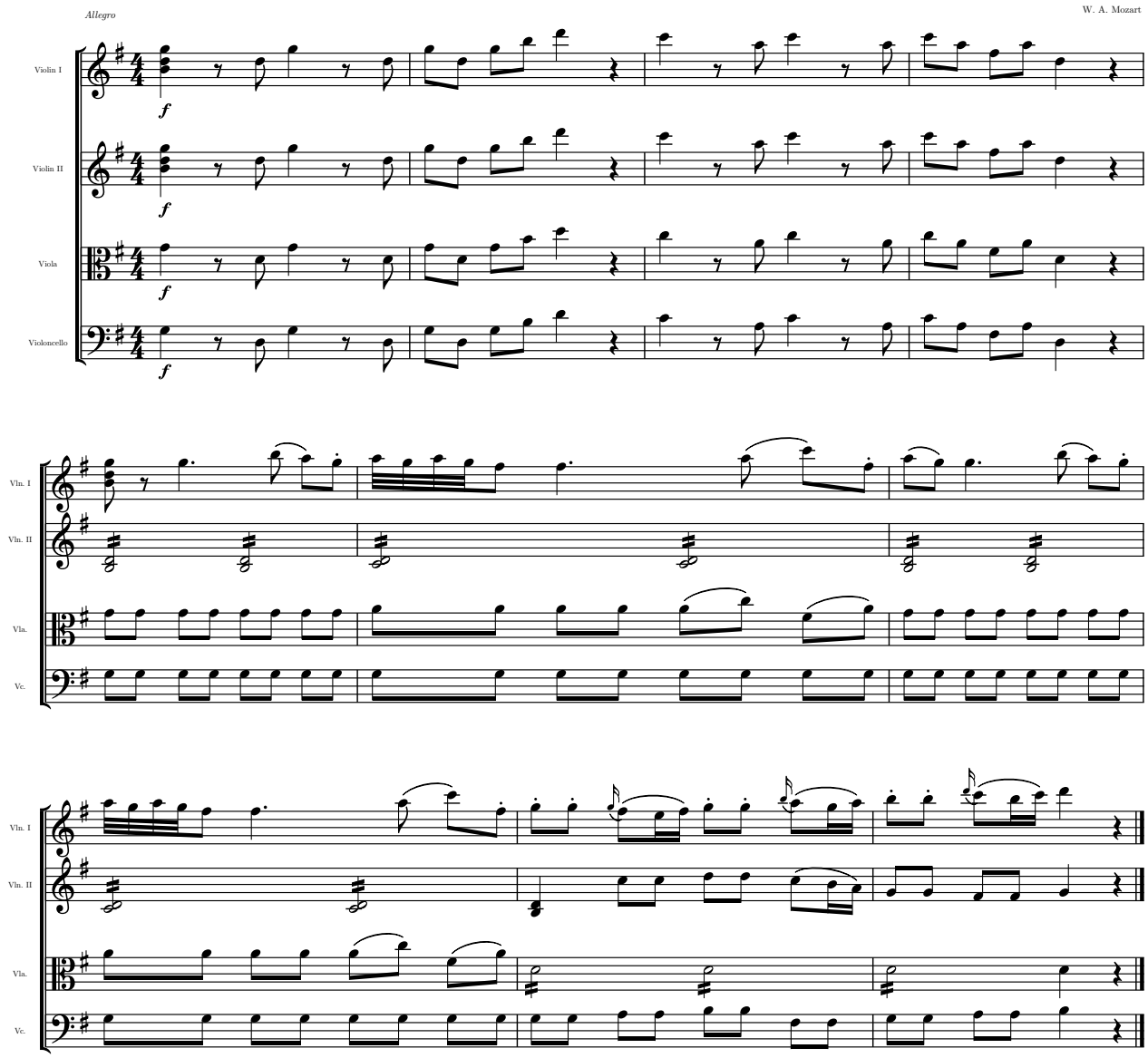
f
Ped.
Ped.
Ped.
Ped.
Ped.
Ped.
Ped.

8 Release showcase
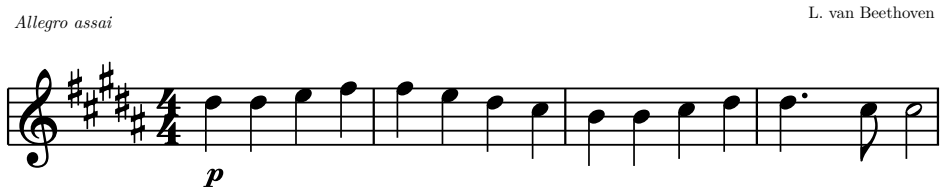
The repository's `examples/` directory contains six reusable, source-checked fixtures and a compiled `showcase.pdf`:

- Chopin, Nocturne Op. 9 No. 2 — piano, pickup and measures 1–4;
- Mozart, Eine kleine Nachtmusik K. 525/I — four-staff strings, measures 1–4;
- Bach, Cello Suite No. 1 BWV 1007/I — solo cello, measures 1–4;
- Beethoven, the four-bar Ode to Joy theme — E-flat alto saxophone, typed at concert pitch;
- Beethoven, Für Elise WoO 59 — piano, pickup and measures 1–4; and
- Beethoven, Piano Sonata Op. 27 No. 2/III — piano, cross-staff coda, measures 178–185.

The saxophone fixture is typed in Beethoven's D major and drawn in B major with `transpose: "M6"` . The others retain their reference score's written pitches. This compact quartet excerpt demonstrates a bracketed ensemble:



The single-staff case needs no staff declaration, even for a transposing instrument written from concert pitch:


Allegro assai
L. van Beethoven
p

9 Limitations

9.1 Current scope

- One to four independent rhythmic voices per staff are supported; the count for a staff is fixed across its bars.
- Cross-staff beams and split chords join two neighboring staves; stems cannot be flipped by hand, and ties cannot cross staves.
- Grace groups currently exclude rests, tuplets, and nested ornamental groups.
- Arpeggio signs span one chord; a cross-staff arpeggio is written on a split chord.
- Pedals and hairpins are event-anchored and do not split automatically at a system break.
- Ottava spans require pitched notation and stay on their voice's staff; end one before a staff switch. Put octave markers on the notation source when mirroring tab.
- Tablature cannot show notehead maps or mirror unpitched percussion.
- Figured bass is drawn under the bottom staff, without continuation lines.
- Percussion staves have five lines; noteheads may be round, X, or circled X.
- Independent tab rejects notation annotations it cannot draw; place markings on its notation source staff. Open-string pitches must use octaves −1 through 9.
- Tab staves draw fret numbers only: no rhythm stems, bends, slides, hammer-ons, or harmonics yet. Automatic fretting takes the lowest position for each onset rather than planning hand positions across a phrase; use `string=` where a passage should stay in one position.
- A chord cannot repeat a written pitch, so shapes that double a pitch on two strings (such as ukulele F or G) need one of the doubled notes left out.
- Dense editorial markings or lyric verses may require `staff-gap`, `note-spacing`, `lyric-gap`, `verse-gap`, or `scale` adjustments.

9.2 Grammar at a glance

Text	Meaning
`c4:q` / `c4q`	Note with an explicit octave and duration.
`c4:q d e f`	Relative pitches inheriting the quarter duration.
`(a4 c e):h`	Relative chord.
`r:q` / `_`	Written rest / computed remainder rest.
`s:q`	Invisible spacer that fills time.
`@lower`	Draw the following events on another staff until the bar ends.
`clef treble`	Change the home staff's clef before the next event.
`(c3 @upper e4):h`	Split chord across two neighboring staves.
`~`	Tie the preceding event.
`/` / `-`	Break / force a local beam connection.
`tuplet 3:2 { c:e d e }`	Three written eighths in the time of two.
`tuplet 3:2[number=never] { c:e d e }`	Tuplet with its numeral hidden.
`cue { c:e d }`	Small notes that still occupy bar time.
`acciaccatura { d:e } f:q`	Slashed single grace note resolving to F.
`tremolo 16 { c:h g:h }`	Two-note alternating tremolo.

`(c e g):h[arpeggio=up]`	Upward arpeggio over a chord.
`lyrics: "Twin -- kle __ _"`	Syllables with a hyphen, melisma extender, and skip.
`[stacc dyn=pp]`	One annotation block with multiple marks.
`barline: (left: "repeat-start")`	Repeat sign on a bar edge.
`barline: (right: "final")`	Final barline on the right edge.
`clef: "bass"`	Persistent single-staff clef change at a bar boundary.
`clef: "percussion"`	Unpitched staff read in treble-clef positions.
`(g5[x] c5):q`	Chord with an X head on one pitch.
`heads: (g5: "x")`	Staff drum map giving a pitch its notehead shape.
`[8va(] … [8va)]`	Ottava span over sounding pitches.
`figures: "(6 4):q 5:h"`	Timed figured bass under the bottom staff.
`clef: "treble-8"`	Guitar clef: treble, sounding an octave lower.
`tab: (clef: "tab", source: "guitar")`	Tab staff repeating the guitar staff's music.
`(c3 e3 g3):h[string=5,4,3]`	Choose tab strings, in written pitch order.
`transpose: "M2"`	Transpose pitched notation, keys, and harmony up a major second.
`part(arguments, "viola")`	Extract one independent staff's score arguments.
`chord-diagram("x32010")`	Standalone chord diagram; `guitar-chords` supplies standard shapes.
`import-score(read("song.mxl", encoding: none))`	Engrave an import with no conversion losses.
`read-score(read("song.abc"))`	Inspect score arguments, title, conversion warnings, and editable source.
`rehearsal: "A"` / `navigation: "segno"`	Boundary rehearsal / navigation mark.
`ending: (label: "Final", start: true)`	Literal volta label.

9.3 License and credits

Package code is released under the MIT License. Bundled Bravura glyphs are released under the SIL Open Font License 1.1. `typed-scores` uses CeTZ for drawing and Bravura's SMuFL metadata for music-glyph geometry.

The Chopin, Mozart, Bach, and Beethoven compositions used by the release fixtures are public domain. Their reusable Typst sources live in the `examples/` directory and are shared by the guide, showcase, and visual regression suite.